

SIMSEP

Sistema de Monitoramento e Controle de Segurança em Pedreiras

Apresentação da 1ª versão do produto — Práticas Interdisciplinares

Gabriel Henrique Damaceno Alves

Universidade Estadual de Goiás — UEG Trindade · Sistemas de Informação

Por que o SIMSEP existe

Pedreiras realizam detonações que geram vibração, ruído e poeira — riscos para trabalhadores e para a comunidade vizinha.

O problema real

O controle de medições e alertas é feito de forma manual e dispersa (planilhas, anotações). Não há histórico centralizado, nem comunicação transparente com os moradores do entorno sobre detonações programadas e níveis de segurança.

Product Goal

Centralizar o monitoramento de segurança da pedreira — pontos, detonações, medições e alertas automáticos — e oferecer um canal público de transparência para a comunidade.

Organização atendida e usuários

- **Organização:** pedreira (empresa de mineração de agregados)
- **Operador de Segurança:** cadastra pontos, detonações e lança medições
- **Engenheiro de Minas:** consulta histórico e analisa riscos
- **Administrador:** gerencia usuários e acompanha indicadores
- **Morador / Comunidade:** consulta painel público, sem login

Histórias de usuário priorizadas

Backlog consolidado após a Sprint 4 — 8 histórias concluídas, 1 parcial, 2 pendentes.

ID	HISTÓRIA (RESUMO)	PRIOR.	EST.	STATUS
US01	Como Administrador, quero realizar login no sistema	Alta	3	Concluída (Sprint 1)
US02	Como Operador, quero cadastrar pontos de monitoramento	Alta	3	Concluída (Sprint 1)
US03	Como Operador, quero cadastrar detonações programadas	Alta	5	Concluída (Sprint 1)
US04	Como Operador, quero registrar medições de vibração e risco de desmoronamento	Alta	5	Concluída (Sprint 2)
US05	Como Operador, quero registrar medições de qualidade do ar e ruído	Alta	5	Concluída (Sprint 2)
US06	Como Operador, quero alertas automáticos quando os limites forem ultrapassados	Alta	8	Concluída (Sprint 2)
US07	Como Operador, quero emitir boletins periódicos informativos	Média	5	Parcial (consulta em tempo real)
US08	Como Morador, quero um canal público sem login para consultar boletins e alertas	Alta	8	Concluída (Sprint 3)
US09	Como Engenheiro de Minas, quero consultar o histórico de medições por período	Média	5	Pendente
US10	Como Administrador, quero visualizar relatórios gerenciais básicos	Baixa	8	Pendente
US11	Como Administrador, quero cadastrar usuários pela interface (sem cadastro público)	Alta	3	Concluída (Sprint 4)

Evolução por Sprint

Cada Sprint selecionou as histórias de maior prioridade ainda não entregues.

Sprint 1

Fundação

- US01 Login com JWT
- US02 Pontos de monitoramento
- US03 Detonações programadas
- Schema do banco e arquitetura

Sprint 2

Medições e alertas

- US04 Medições de vibração
- US05 Ar e ruído
- US06 Alertas automáticos por limite
- Painel (dashboard) com indicadores

Sprint 3

Transparência

- US08 Painel público sem login
- US07 Boletim em tempo real (parcial)
- Rotas públicas somente leitura

Sprint 4

Segurança de acesso

- US11 Cadastro de usuários restrito ao Admin
- Bloqueio do registro público (HTTP 403)
- Middleware RBAC em /api/usuarios
- Migração bcrypt → bcryptjs (npm audit)

Critérios objetivos e verificáveis

Exemplos de histórias com os critérios usados para considerá-las concluídas.

US06 · Alertas automáticos

- Alerta criado **na mesma transação** ao gravar medição acima do limite
- Mensagem clara com valor, unidade e limite
- Registro de data/hora e vínculo com a medição
- Alerta pode ser marcado como **resolvido**

US08 · Canal público

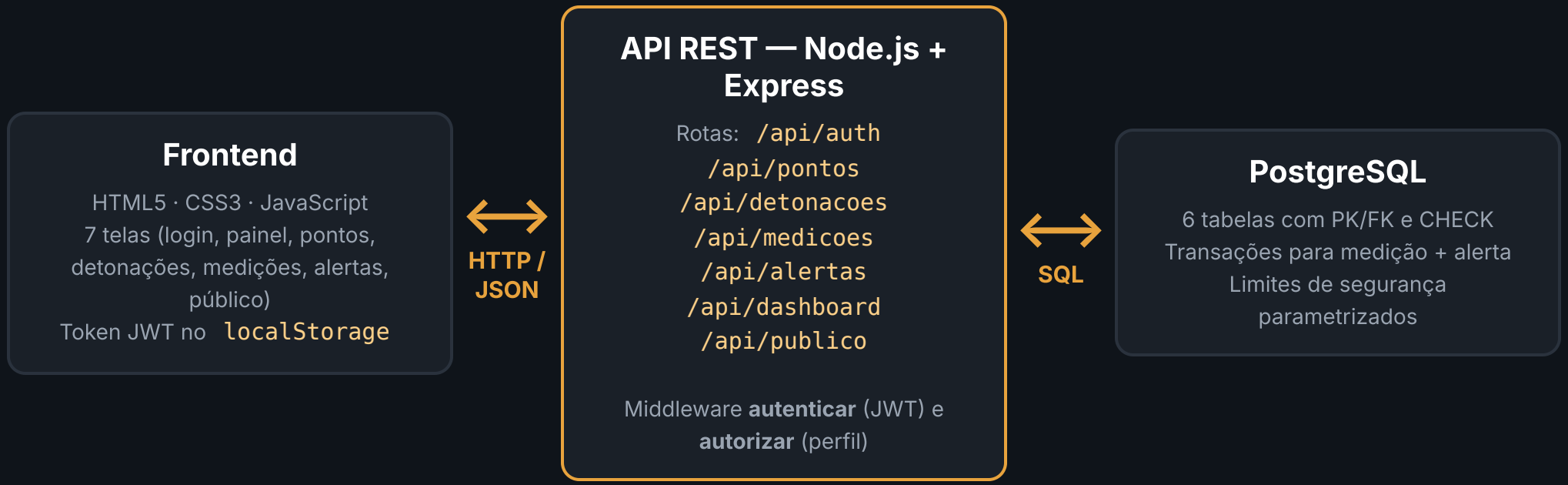
- Acesso **sem login**
- Somente leitura (apenas GET)
- Sem exposição de dados sensíveis (usuários, e-mails)
- Mostra medições atuais, detonações futuras e alertas

US11 · Cadastro de usuários

- Registro público bloqueado após o 1º acesso
- Apenas perfil **admin** cadastra usuários
- Admin não exclui a própria conta
- Não é possível excluir o último admin

Arquitetura em três camadas

Frontend estático consome uma API REST, que persiste em banco relacional.



CAMADA	TECNOLOGIA	JUSTIFICATIVA
Frontend	HTML5, CSS3, JavaScript puro	Tecnologias livres, dominadas pela equipe e suficientes para o MVP
Backend	Node.js + Express	Framework leve, curva de aprendizado rápida para API REST
Banco	PostgreSQL	SGBD relacional gratuito e robusto
Autenticação / Autorização	JWT + bcryptjs · middleware RBAC	Sessão sem estado; senhas com hash; rotas administrativas restritas
Versionamento	Git / GitHub	Histórico de commits por funcionalidade

5 · BANCO DE DADOS

Modelo relacional

PK chave primária · **FK** chave estrangeira · restrições UNIQUE, NOT NULL e CHECK garantem a integridade.

usuarios

id SERIAL
nome, email *UNIQUE*
senha_hash
perfil CHECK (operador | engenheiro | admin)
criado_em

pontos_monitoramento

id SERIAL
nome *UNIQUE*
localizacao
descricao

limites_seguranca

id SERIAL
tipo *UNIQUE* CHECK (vibracao | ruido | ar)
valor_limite NUMERIC
unidade
5 mm/s · 85 dB · 50 µg/m³

detonacoes

id SERIAL
identificador *UNIQUE*
data_hora
ponto_id → pontos_monitoramento
responsavel_id → usuarios

medicoes

id SERIAL
ponto_id → pontos_monitoramento
detonacao_id → detonacoes (opcional)
tipo CHECK, valor NUMERIC, unidade
data_hora DEFAULT now()

alertas

id SERIAL
medicao_id → medicoes
mensagem
status CHECK (aberto | resolvido)
data_hora DEFAULT now()

6 · FUNCIONALIDADES IMPLEMENTADAS

O que o sistema faz hoje

Sistema executado com banco PostgreSQL real; dados de exemplo cadastrados pela própria interface/API.

4
Pontos de monitoramento

3
Detonações registradas

8
Medições lançadas

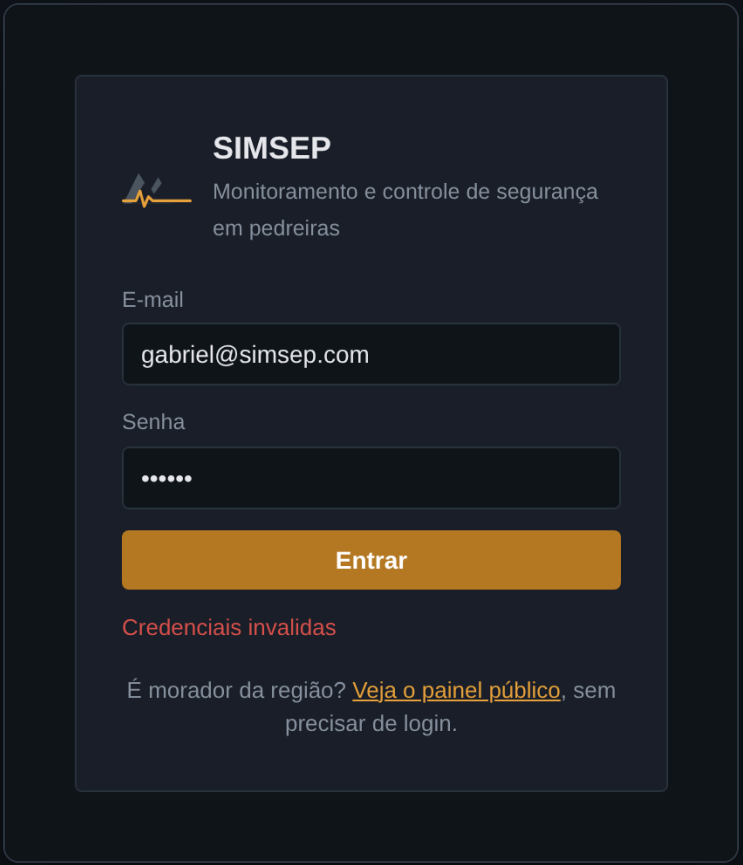
3
Alertas gerados automaticamente

- **Login** com e-mail/senha, hash de senha e token JWT (8h)
- **Pontos de monitoramento**: cadastro, edição e exclusão
- **Detonações**: identificador único, data/hora, ponto e responsável
- **Medições** de vibração, ruído e ar, com filtro por período
- **Alertas automáticos** quando a medição ultrapassa o limite configurado
- **Resolução de alertas** pelo operador
- **Painel** com indicadores e últimos alertas em aberto
- **Painel público** para a comunidade, sem login

Login e tratamento de erro

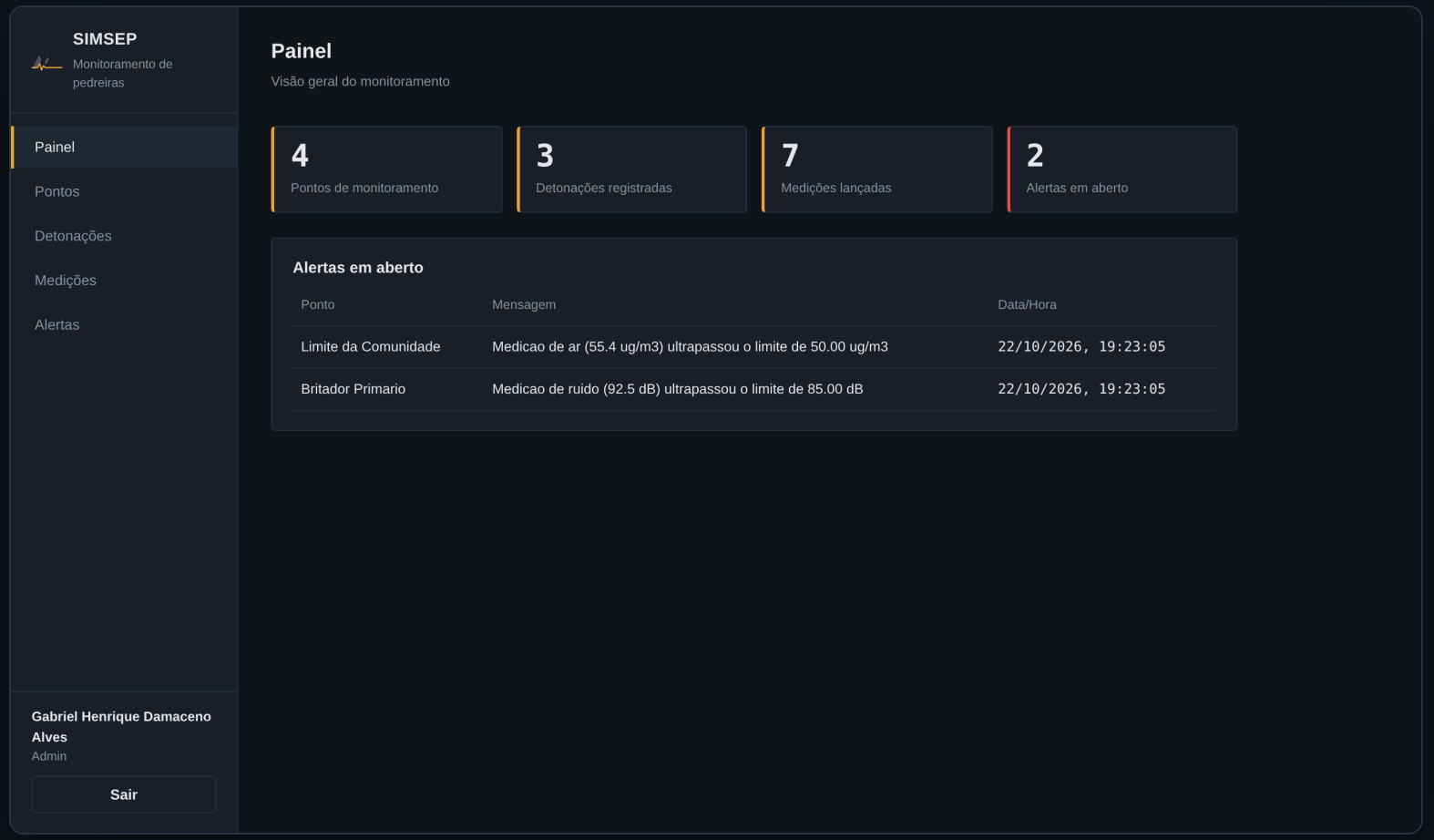


Tela de login, com link para o painel público da comunidade.



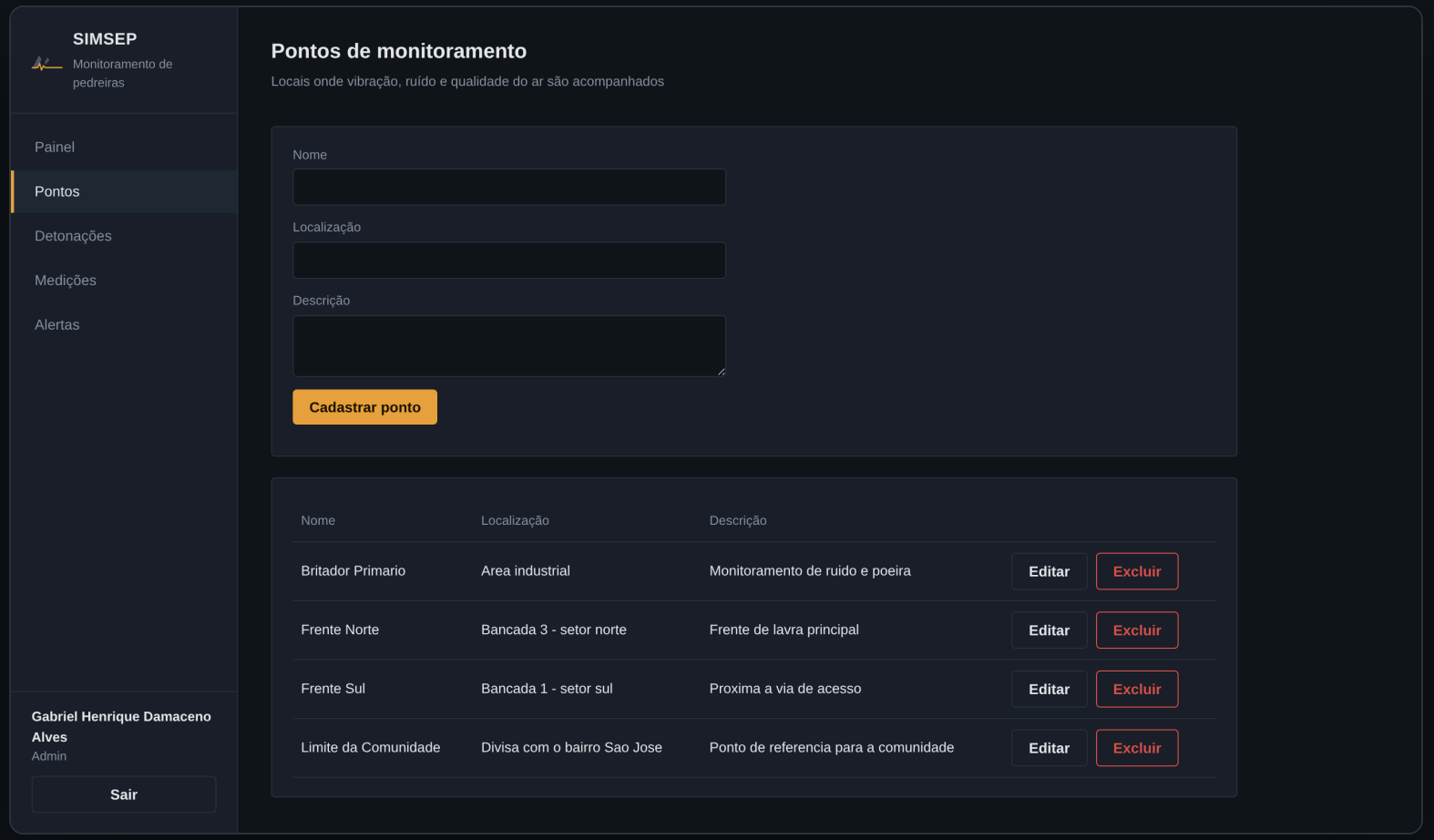
Credenciais inválidas: a API responde 401 e a tela exibe a mensagem, sem revelar se o e-mail existe.

Painel do administrador



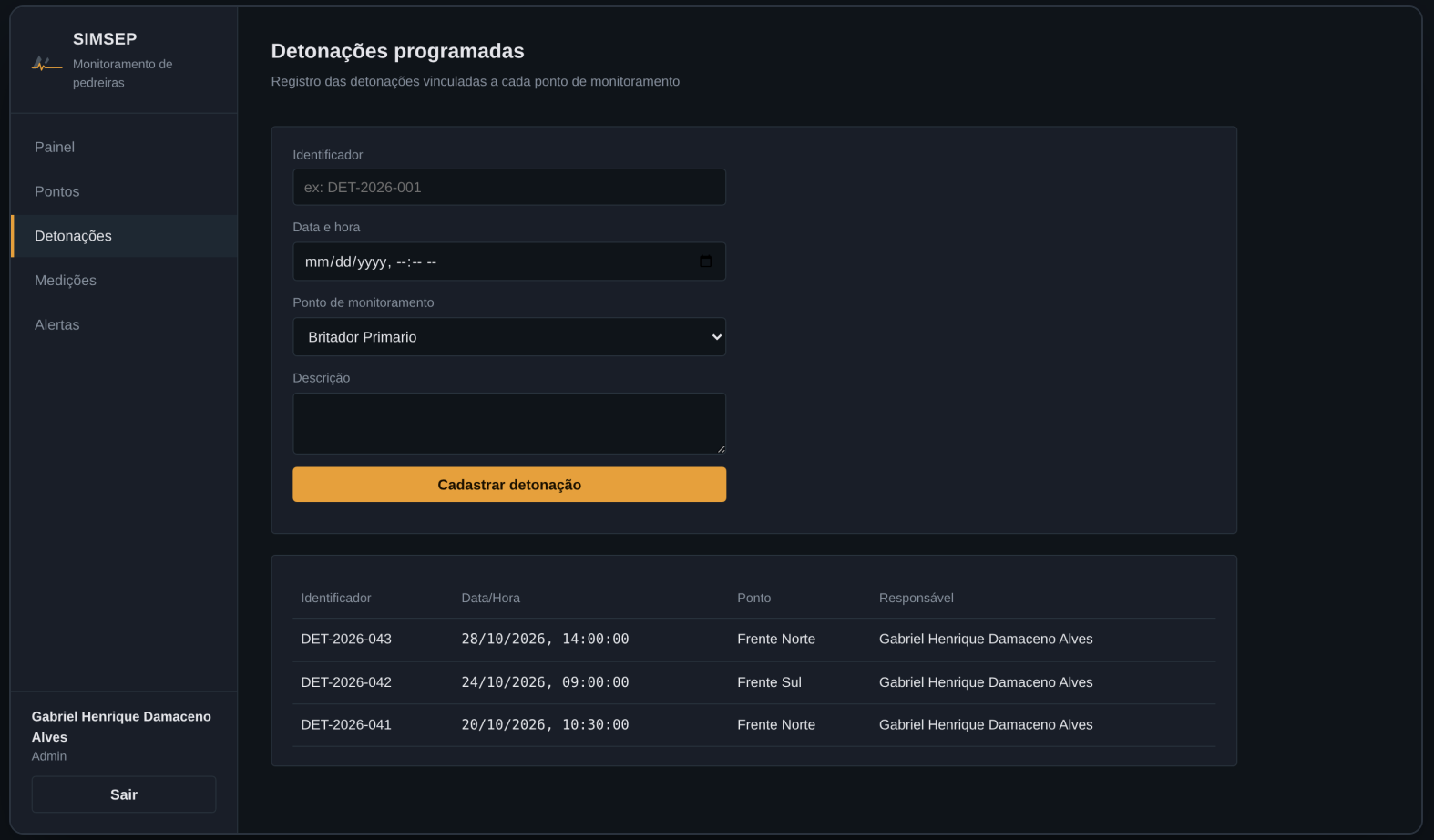
Contadores vindos de `GET /api/dashboard/resumo` e os alertas em aberto mais recentes.

Pontos de monitoramento



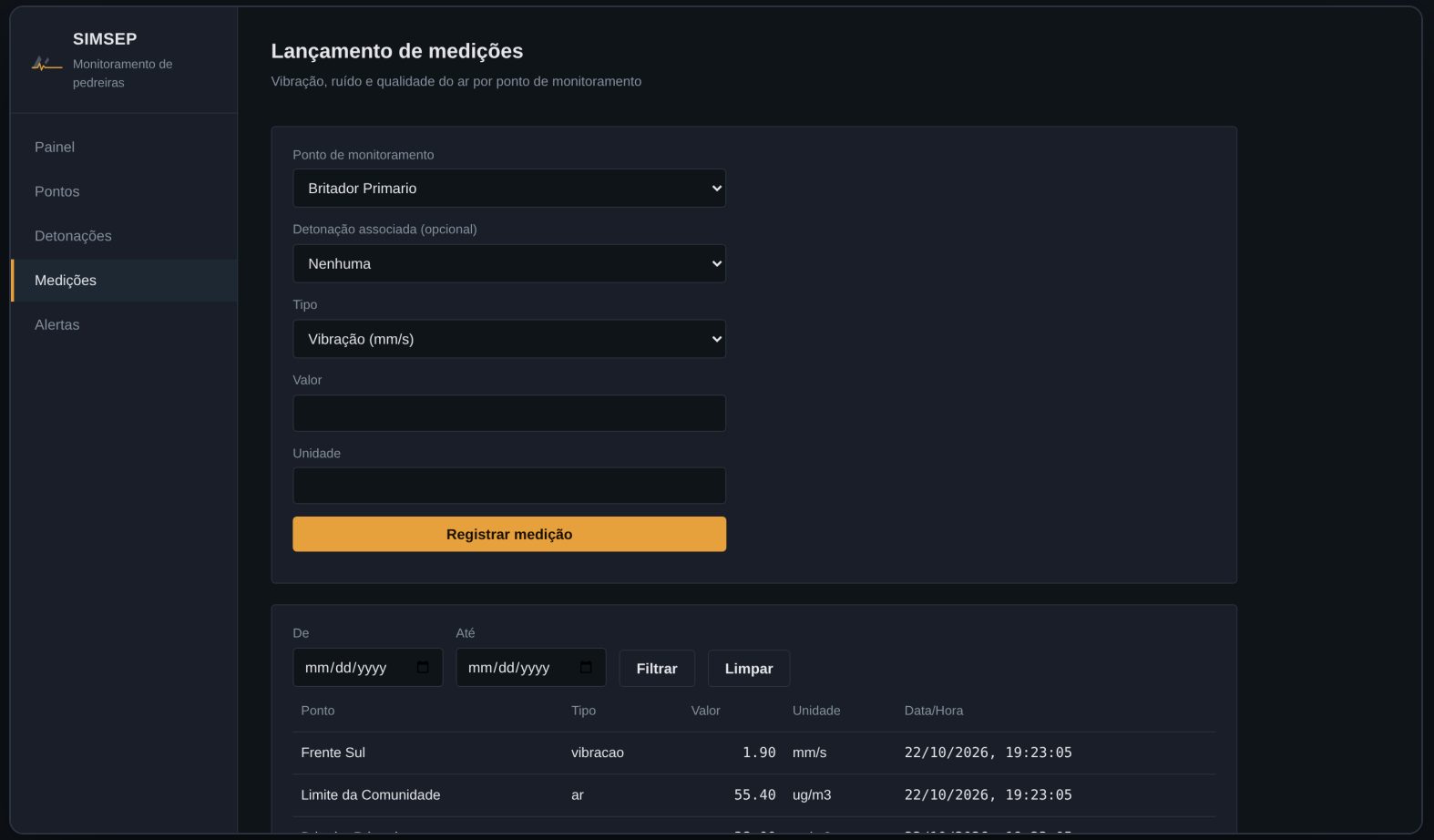
CRUD completo — nome único validado pelo banco (UNIQUE) e pela API (HTTP 409 em duplicidade).

Detonações programadas



Cada detonação é associada a um ponto e ao usuário responsável (extraído do token JWT).

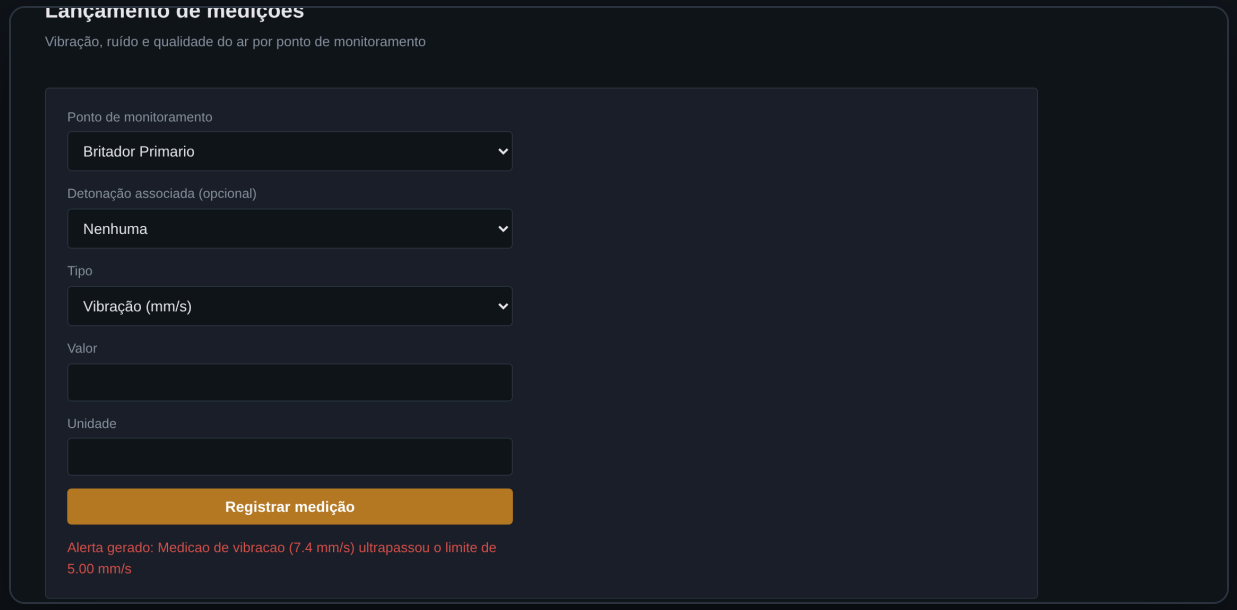
Lançamento de medições



Vibração, ruído e qualidade do ar por ponto, com vínculo opcional a uma detonação e filtro por período.

7 · REGRA DE NEGÓCIO INTEGRADA

Medição acima do limite → alerta automático



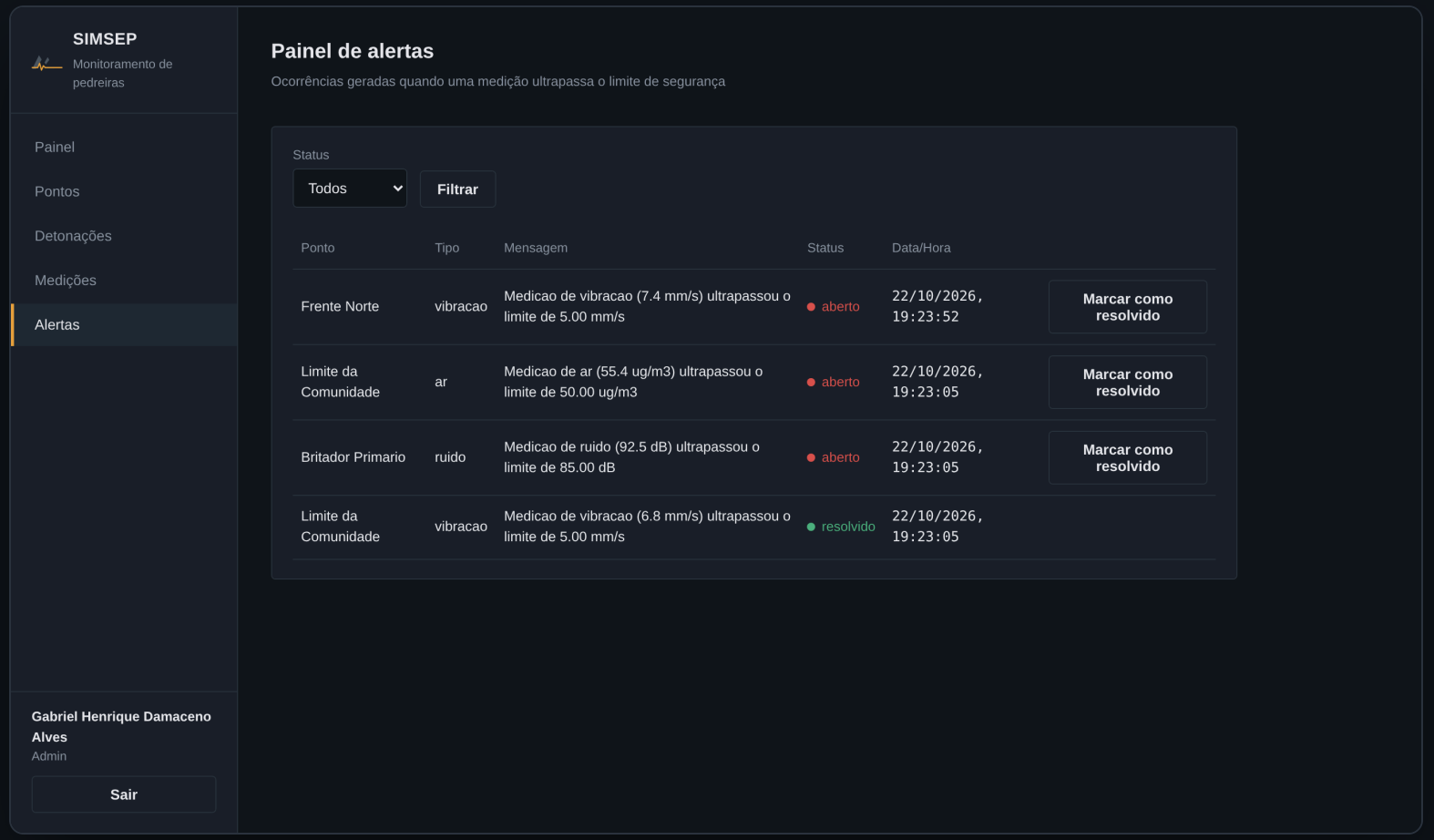
Vibração de 7,4 mm/s na Frente Norte: a API gravou a medição e gerou o alerta na mesma transação.

Como funciona

1. `BEGIN` — abre a transação
2. `INSERT` na tabela **medicoes**
3. Consulta **limites_seguranca** pelo tipo
4. Se `valor > valor_limite` : `INSERT` em **alertas** com mensagem descritiva
5. `COMMIT` — ou `ROLLBACK` em caso de erro

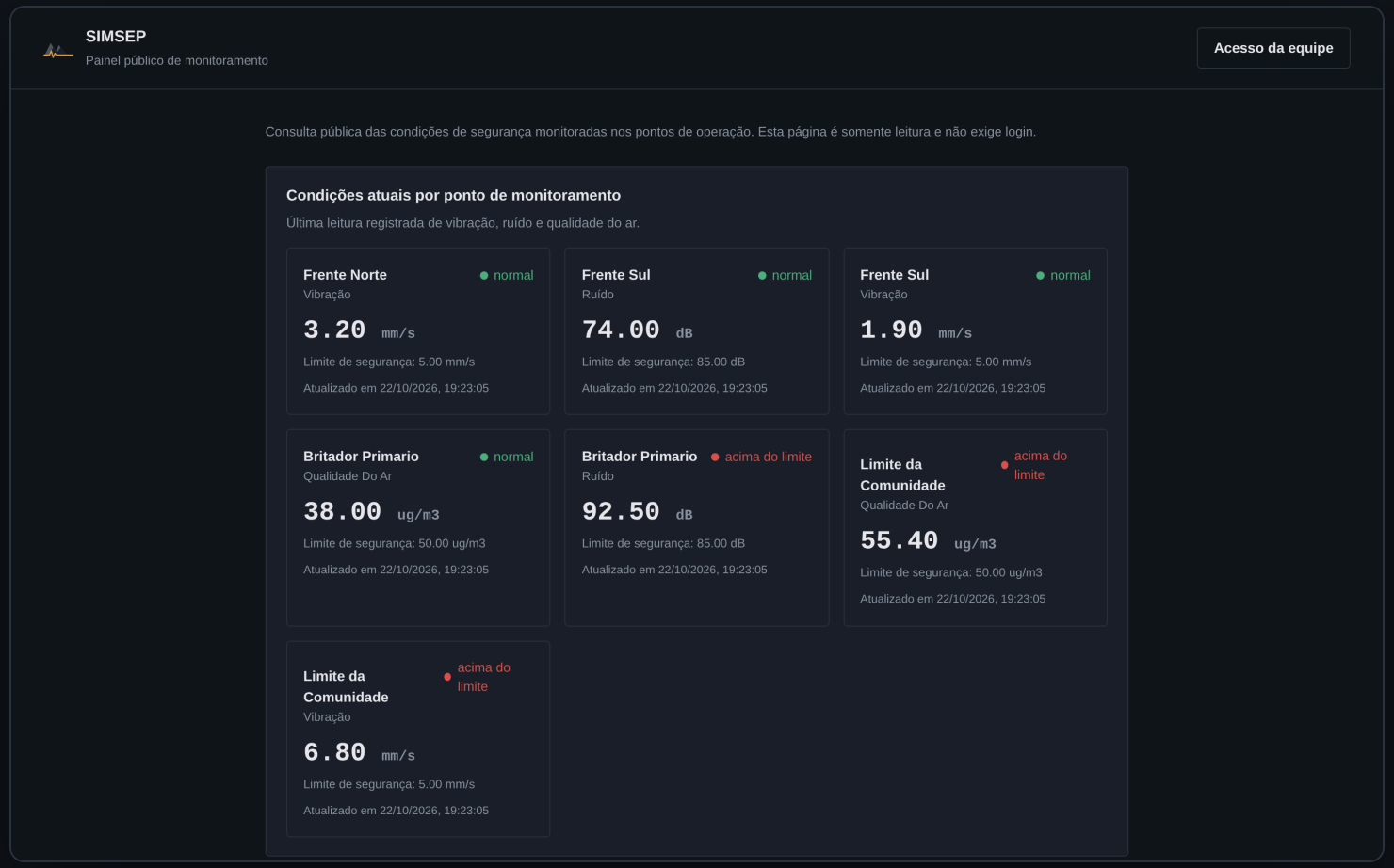
Garante que nunca exista medição crítica sem alerta correspondente.

Gestão de alertas



Filtro por status (aberto / resolvido) e ação de resolver via `PUT /api/alertas/:id/resolver`.

Painel público da comunidade



Sem login e somente leitura: situação atual de cada ponto (normal / acima do limite), detonações futuras e últimos alertas.

SPRINT 4 · SEGURANÇA

Autenticação e autorização

Uma não conformidade encontrada no uso real virou a história US11.

Problema identificado

`POST /api/auth/registrar` estava aberto: qualquer pessoa podia criar uma conta **admin**.

Correção

- Registro público só funciona enquanto não há usuários (cria o 1º admin); depois responde **403**
- Novas rotas `GET/POST/DELETE /api/usuarios` restritas ao perfil admin
- Tela de usuários visível apenas no menu de administradores

Camadas de proteção

- **autenticar**: exige `Authorization: Bearer <JWT>` — 401 sem token, 403 token inválido
- **autorizar(perfis)**: RBAC simples por perfil (operador, engenheiro, admin)
- Senhas nunca em texto puro — hash com bcryptjs
- Regras de negócio: admin não se autoexclui nem exclui o último admin
- `npm audit` incorporado à checklist de cada Sprint

Testes realizados

Testes manuais e via API, verificando os critérios de aceitação e o tratamento de erros.

CENÁRIO TESTADO	RESULTADO ESPERADO	OBTIDO
Login com senha incorreta	401 "Credenciais invalidas"	✔ 401
Acesso a `/api/pontos` sem token	401 "Token nao informado"	✔ 401
Token inválido / adulterado	403 "Token invalido"	✔ 403
Operador acessa `/api/usuarios` (Sprint 4)	403 "Perfil sem permissao"	✔ 403
Registro público com usuários já existentes (Sprint 4)	403 bloqueado	✔ 403
Medição sem valor/unidade	400 campos obrigatórios	✔ 400
Ponto com nome duplicado	409 conflito (UNIQUE)	✔ 409
Medição de vibração 6,8 mm/s (limite 5)	Alerta criado automaticamente	✔ alerta #1
Medição de vibração 3,2 mm/s	Nenhum alerta	✔ alerta = null
Resolver alerta	status = resolvido	✔
Admin tenta excluir a própria conta / último admin (Sprint 4)	Rejeitado pela regra de negócio	✔

Lições aprendidas e roadmap

O que funcionou bem

- Falha de segurança identificada de forma proativa, antes de virar incidente
- Padrão JWT existente facilitou o middleware de autorização
- Transação medição + alerta manteve a consistência dos dados

Dificuldades

- Troca de bcrypt por bcryptjs exigiu reinstalação cuidadosa
- Casos-limite de exclusão de administradores não previstos no backlog

Próximas Sprints

- **US09** — histórico de medições por período para o Engenheiro de Minas, com exportação
- **US10** — relatórios gerenciais para o Administrador
- **US07** — boletim periódico exportável (PDF) para a comunidade
- Testes automatizados da API (Jest + Supertest)
- Rodar `npm audit` a cada Sprint antes do merge

PARTE II · COMPLEMENTO SOLICITADO NA ENTREGA 1

9–13

Artefatos de engenharia do SIMSEP

Documento de requisitos · Histórias de usuário com critérios de aceitação · Desenho do banco de dados · Diagrama e especificação de casos de uso · Tipos de testes e rastreabilidade

Os artefatos também estão versionados na pasta `docs/` do repositório.

Gabriel Henrique Damaceno Alves · UEG Trindade

Requisitos funcionais

Escopo: monitoramento de vibração, ruído e qualidade do ar em pedreiras, com alertas e transparência à comunidade. Atores: Operador de Segurança, Engenheiro de Minas, Administrador, Morador.

ID	REQUISITO FUNCIONAL	PRIOR.	HISTÓRIA
RF01	Autenticar usuário por e-mail e senha, emitindo token JWT com validade de 8 h	Alta	US01
RF02	Manter pontos de monitoramento (cadastrar, editar, excluir, listar), com nome único	Alta	US02
RF03	Registrar detonações programadas com identificador único, data/hora, ponto e responsável	Alta	US03
RF04	Registrar medições de vibração, ruído e qualidade do ar por ponto, com vínculo opcional a uma detonação	Alta	US04, US05
RF05	Gerar alerta automaticamente, na mesma transação, quando a medição ultrapassar o limite de segurança do tipo	Alta	US06
RF06	Permitir marcar alerta como resolvido, registrando data e responsável	Alta	US06

ID	REQUISITO FUNCIONAL	PRIOR.	HISTÓRIA
RF07	Exibir painel com indicadores e alertas em aberto para usuários autenticados	Média	US06
RF08	Disponibilizar painel público, sem login e somente leitura, com situação atual, detonações futuras e alertas	Alta	US08
RF09	Emitir boletins periódicos consolidando medições, detonações e alertas de um período, publicados à comunidade	Média	US07
RF10	Consultar histórico de medições por período, ponto e tipo, com estatísticas e exportação CSV	Média	US09
RF11	Gerar relatórios gerenciais por período (totais, alertas por tipo/ponto, % resolvidos, tempo de resolução)	Média	US10
RF12	Gerenciar usuários e perfis exclusivamente pelo Administrador; cadastro público apenas para o 1º acesso	Alta	US11

Requisitos não funcionais e regras de negócio

Restrições de qualidade que o sistema deve atender e regras do domínio que valem independentemente da tela.

ID	REQUISITO NÃO FUNCIONAL	CATEGORIA
RNF01	Senhas armazenadas apenas com hash bcrypt (custo 10); nunca em texto puro	Segurança
RNF02	Rotas internas exigem `Authorization: Bearer`; 401 sem token, 403 token inválido ou perfil sem permissão	Segurança
RNF03	Dados vindos da API são escapados antes de ir ao HTML (proteção contra XSS)	Segurança
RNF04	Painel público não expõe e-mails, nomes de usuários nem identificadores internos	Privacidade
RNF05	Integridade garantida no banco: PK/FK, UNIQUE, CHECK e transações para medição + alerta	Confiabilidade
RNF06	Consultas de histórico e relatório respondem em < 1 s para até 100 mil medições (índices por data, ponto e tipo)	Desempenho
RNF07	Toda resposta da API é JSON, inclusive erros (campo `erro`) e rotas inexistentes (404)	Usabilidade/API
RNF08	Regras de negócio cobertas por testes automatizados executáveis com `npm test`	Manutenibilidade
RNF09	Tecnologias livres: Node.js + Express, PostgreSQL, HTML/CSS/JS; sem dependências pagas	Portabilidade

ID	REGRA DE NEGÓCIO
RN01	Limites de segurança por tipo: vibração 5 mm/s · ruído 85 dB · ar 50 µg/m³ (parametrizados em `limites_seguranca`)
RN02	Medição com valor **maior** que o limite gera exatamente um alerta *aberto*; valor igual ao limite não gera
RN03	Valor de medição não pode ser negativo; unidade é definida pelo tipo (mm/s, dB, µg/m³)
RN04	Alerta só pode ser resolvido uma vez; a resolução registra quem e quando
RN05	Ponto com medições ou detonações vinculadas não pode ser excluído
RN06	Admin não exclui a própria conta nem o último administrador; cadastro público só enquanto não há usuários
RN07	Perfis: Operador e Admin lançam dados e emitem boletins; Engenheiro apenas consulta; Admin gerencia usuários e relatórios
RN08	Boletim emitido guarda um retrato dos dados do período e não muda após a publicação

Histórias com critérios de aceitação (1/2)

Formato: “Como <perfil>, quero <ação> para <benefício>”, com critérios objetivos e verificáveis por teste.

US01 · Login Concluída

Como **Administrador**, quero realizar login para acessar o sistema de forma segura.

Critérios: e-mail/senha válidos retornam token JWT (8 h) e perfil · senha errada ou e-mail inexistente respondem 401 “Credenciais invalidas” sem revelar qual campo errou · campos vazios respondem 400.

US02 · Pontos de monitoramento Concluída

Como **Operador**, quero cadastrar pontos de monitoramento para organizar onde as medições são feitas.

Critérios: nome e localização obrigatórios (400) · nome único (409) · editar e excluir · exclusão bloqueada com dados vinculados (409) · ponto inexistente 404.

US03 · Detonações programadas Concluída

Como **Operador**, quero cadastrar detonações programadas para informar a equipe e a comunidade.

Critérios: identificador único (409) · data/hora válida · ponto existente · responsável extraído do token · detonações futuras aparecem no painel público.

US04/US05 · Medições Concluída

Como **Operador**, quero registrar medições de vibração, ruído e qualidade do ar para acompanhar a segurança da operação.

Critérios: ponto, tipo e valor obrigatórios · tipo ∈ {vibracao, ruido, ar} · valor numérico ≥ 0 · unidade padronizada pelo tipo · vínculo opcional a detonação · Engenheiro não lança (403).

US06 · Alertas automáticos Concluída

Como **Operador**, quero alertas automáticos quando os limites forem ultrapassados para agir rapidamente.

Critérios: alerta criado na mesma transação da medição · mensagem com valor, unidade, limite e % excedido · pode ser resolvido uma única vez, registrando quem/quando · 404 para alerta inexistente.

US07 · Boletins periódicos Concluída · Sprint 1

Como **Operador**, quero emitir boletins periódicos para informar a comunidade sobre as condições do período.

Critérios: escolher período e pré-visualizar · resumo gerado automaticamente · emissão grava retrato dos dados · boletim listado no painel público e imprimível · Engenheiro não emite (403).

Histórias com critérios de aceitação (2/2)

Histórias de transparência, análise e administração — incluindo as entregues na Sprint 1 e as já priorizadas para a próxima.

US08 · Canal público Concluída

Como **Morador**, quero um canal público sem login para consultar boletins e alertas da pedreira.

Critérios: acesso sem token · somente GET · mostra última leitura por ponto/tipo com situação normal/acima do limite, detonações futuras e últimos alertas · nenhum dado de usuário exposto.

US09 · Histórico por período Concluída · Sprint 1

Como **Engenheiro de Minas**, quero consultar o histórico de medições por período para analisar tendências e riscos.

Critérios: filtros por início/fim, ponto e tipo · datas inválidas respondem 400 · mín/média/máx e quantidade por ponto e tipo · situação de cada leitura · exportação CSV abrível no Excel.

US10 · Relatórios gerenciais Concluída · Sprint 1

Como **Administrador**, quero visualizar relatórios gerenciais para acompanhar o desempenho da segurança.

Critérios: apenas admin (403 para outros) · período configurável, padrão 30 dias · totais, % de alertas resolvidos, tempo médio de resolução, distribuição por tipo/ponto/semana · imprimível.

US11 · Cadastro de usuários Concluída

Como **Administrador**, quero cadastrar usuários pela interface para que ninguém crie contas sem autorização.

Critérios: registro público só para o 1º admin, depois 403 · apenas admin cria/exclui · validação de e-mail, senha ≥ 6 e perfil · admin não se exclui nem exclui o último admin.

TD01 · Testes automatizados Concluída · Sprint 1

Como **equipe**, queremos testes automatizados da API e das regras de negócio para evitar regressões.

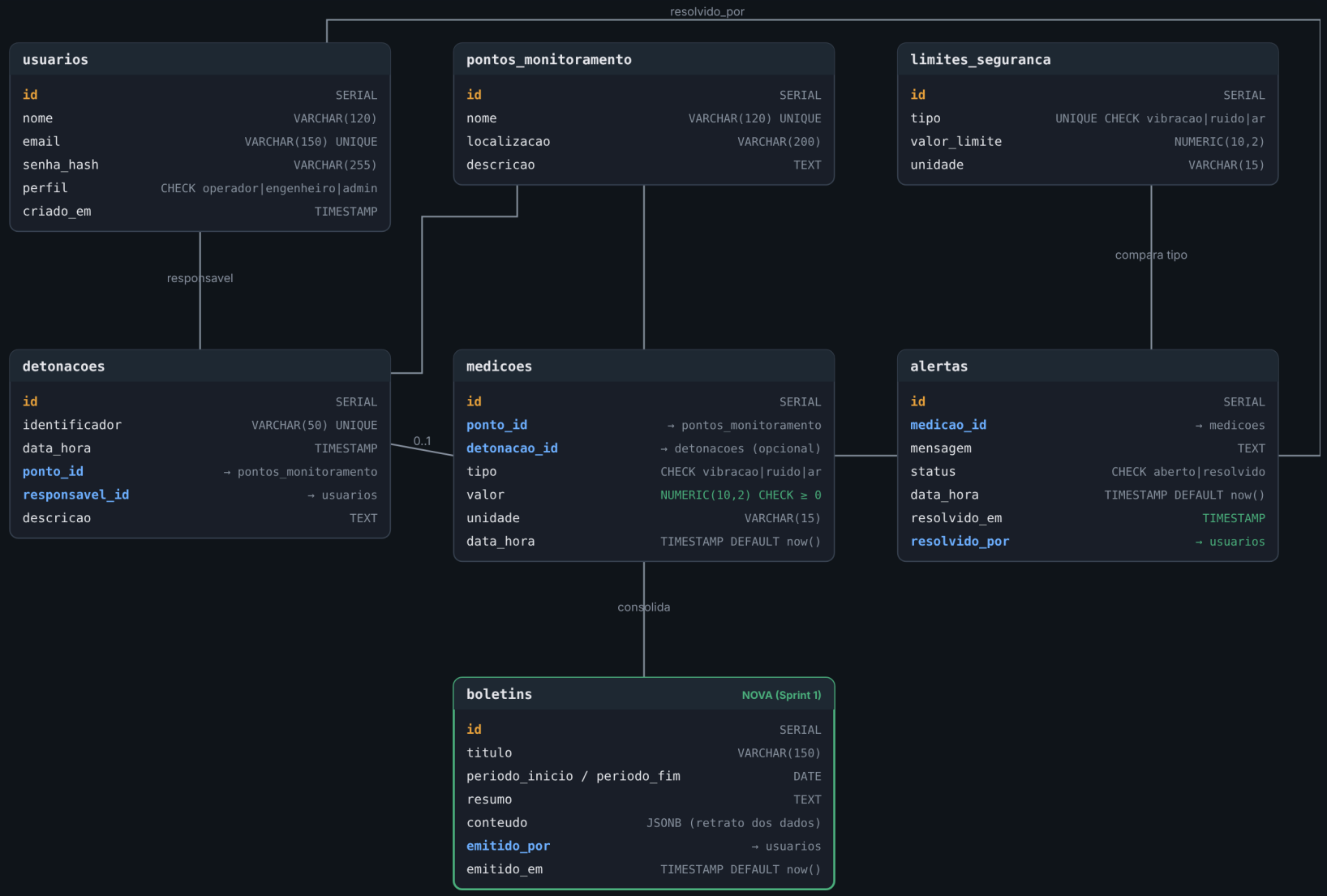
Critérios: `npm test` roda sem banco em < 1 s · suíte de integração cobre autenticação, autorização, transação de alerta, histórico, relatório e boletim · cenários da Entrega 1 viram testes.

US12 · Limites de segurança Próxima Sprint

Como **Administrador**, quero editar os limites de segurança pela interface para ajustá-los sem alterar o banco.

Critérios: apenas admin · valor > 0 · alteração registra quem/quando · novas medições usam o limite vigente; alertas antigos não mudam.

Diagrama entidade-relacionamento (PostgreSQL)



PK chave primária · **FK** chave estrangeira · **verde** adicionado na Sprint 1 (19–25/10) · notação pé-de-galinha: 1 — N

Dicionário de dados e integridade

Sete tabelas; toda regra crítica existe também no banco, não só na API.

TABELA	PROPÓSITO	CHAVES E RELACIONAMENTOS	RESTRIÇÕES QUE PROTEGEM O DOMÍNIO
`usuarios`	Contas com perfil de acesso	PK id	email UNIQUE · perfil CHECK (operador | engenheiro | admin) · senha somente como hash
`pontos_monitoramento`	Locais onde se mede	PK id	nome UNIQUE · localização NOT NULL · RESTRICT ao excluir com dependentes
`limites_seguranca`	Limite por tipo de medição	PK id	tipo UNIQUE CHECK (vibracao | ruido | ar) · valor_limite NUMERIC(10,2)
`detonacoes`	Detonações programadas/realizadas	PK id · FK ponto_id → pontos · FK responsavel_id → usuarios	identificador UNIQUE · data_hora NOT NULL · índice por data_hora
`medicoes`	Leituras de vibração, ruído e ar	PK id · FK ponto_id → pontos · FK detonacao_id → detonacoes (opcional)	tipo CHECK · **valor CHECK ≥ 0** · data_hora DEFAULT now() · índices (data_hora) e (ponto_id, tipo)
`alertas`	Ocorrências acima do limite	PK id · FK medicao_id → medicoes · **FK resolvido_por → usuarios**	status CHECK (aberto | resolvido) DEFAULT aberto · **resolvido_em** · criado na mesma transação da medição
`boletins` NOVA	Boletins emitidos à comunidade	PK id · FK emitido_por → usuarios	periodo_fim ≥ periodo_inicio (CHECK) · conteudo JSONB imutável após emissão · índice por emitido_em

Transação medição + alerta

- `BEGIN` → INSERT medição → consulta limite → INSERT alerta se valor > limite → `COMMIT`
- Erro em qualquer passo → `ROLLBACK`: nunca há medição crítica sem alerta

Evolução do schema

- `sql/schema.sql` — instalação nova completa
- `sql/migracao_sprint1.sql` — idempotente (IF NOT EXISTS) para bancos já em uso

Validação

- Schema e migração aplicados em banco limpo
- CHECK e FK testados com dados inválidos (valor −1, exclusão de ponto com medições)

Atores e casos de uso (UML)



verde = casos de uso implementados na Sprint 1 (19–25/10/2026) · linhas tracejadas = «include» / «extend»

UC04 · Registrar medição (inclui UC05 · Gerar alerta automático)

Caso de uso central do sistema — detalhado como exemplo; os demais seguem o mesmo modelo em `docs/casos-de-uso.md`.

IDENTIFICAÇÃO	
Ator principal	Operador de Segurança (Administrador herda)
Pré-condições	Usuário autenticado com perfil operador/admin; ao menos um ponto cadastrado; limites de segurança configurados
Pós-condições	Medição persistida; se acima do limite, alerta aberto vinculado; painel e painel público refletem a nova leitura
Requisitos	RF04, RF05 · RN01, RN02, RN03
Testes	Unitários `avaliarLimite`, `validarMedicao` · Integração "medicao acima do limite gera alerta", "dentro do limite nao gera", "negativa ... 400", "engenheiro ... 403"

Fluxo principal

- 1. Operador abre "Medições" e seleciona ponto, detonação (opcional) e tipo; a unidade é preenchida automaticamente
- 2. Informa o valor e confirma
- 3. Sistema valida os dados (RN03) e abre uma transação
- 4. Sistema grava a medição e consulta o limite do tipo (RN01)
- 5. Se valor > limite, sistema cria alerta aberto com mensagem descritiva (RN02) — **UC05**
- 6. Sistema confirma a transação e responde 201 com medição e alerta (ou `null`)
- 7. Tela exibe confirmação (verde) ou o alerta gerado (vermelho) e atualiza o histórico

Fluxos alternativos e de exceção

- **A1** Nenhum ponto cadastrado → tela orienta a cadastrar antes; API 400
- **E1** Valor negativo, não numérico ou tipo inválido → 400 com a lista de erros
- **E2** Ponto/detonação inexistente → 400 "nao existem"
- **E3** Perfil Engenheiro → 403, sessão mantida
- **E4** Falha ao gravar alerta → ROLLBACK da medição, 500

Tipos de testes aplicados

Estratégia em camadas: regras puras testadas sem banco, API testada ponta a ponta, banco testado com dados inválidos, telas validadas por roteiro.

TIPO	OBJETIVO	O QUE COBRE NO SIMSEP	FERRAMENTA / EXECUÇÃO	QTDE
Unitário	Verificar uma regra isolada	Validação de medição, usuário e período · avaliação de limite e mensagem do alerta · classificação de situação · CSV · montagem do boletim	`node:test · npm test`	25
Integração (API)	Verificar rotas + banco juntos	Login, 401/403, bloqueio do registro público, RBAC por perfil, CRUD de pontos, transação medição + alerta, histórico/estatísticas/CSV, resolver alerta, relatório, boletim, rotas públicas	`node:test + fetch · npm run test:integracao`	20
Banco de dados	Garantir integridade	Schema e migração em banco limpo · CHECK/FK/UNIQUE rejeitando dados inválidos · consultas agregadas com massa de exemplo	`psql`	9
Funcional / aceitação	Validar critérios das histórias na tela	Roteiro com 10 passos nos perfis operador, engenheiro, admin e público	Manual, navegador	10
Segurança	Procurar brechas	XSS em campos de texto · token ausente/adulterado · escalação de perfil · exposição de dados no público · `npm audit`	Integração + revisão	6
Regressão	Evitar que o que funcionava quebre	Cenários da Entrega 1 (401/403/400/409, alerta automático, regras de admin) automatizados e rodados a cada Sprint	Suíte de integração	—

Pirâmide

- Base larga de unitários (rápidos, sem banco)
- Integração cobre os fluxos de valor
- Poucos testes manuais, só de aceitação

Critério de pronto (DoD)

- Critérios de aceitação verificados
- `npm test` verde e `npm audit` sem vulnerabilidade alta
- Documentação e migração atualizadas

Resultado atual

- **25/25 unitários · 20/20 integração**
- 0 vulnerabilidades altas no `npm audit`

Matriz de rastreabilidade

Cada história ligada aos requisitos que atende, às regras de negócio e aos testes que a verificam.

HISTÓRIA	REQUISITOS	REGRAS	TESTES UNITÁRIOS	TESTES DE INTEGRAÇÃO (API)	STATUS
US01 Login	RF01, RNF01, RNF02	—	—	senha errada 401 · sem token 401 · token adulterado 403	OK
US02 Pontos	RF02, RNF05	RN05	—	cadastra e rejeita duplicado 409 · excluir com medições 409 / inexistente 404	OK
US03 Detonações	RF03	—	—	cobertas via histórico e boletim (vínculo de detonação)	OK
US04/05 Medições	RF04, RNF05	RN03, RN07	validarMedicao (5)	negativa/tipo inválido 400 · engenheiro 403	OK
US06 Alertas	RF05, RF06	RN01, RN02, RN04	avaliarLimite (5) · classificarSituacao (4)	acima do limite gera alerta · dentro não gera · resolver grava resolvido_por, 409 e 404	OK
US07 Boletins	RF09, RNF04	RN07, RN08	montarBoletim (3)	prévia exige período · engenheiro 403 · emissão grava e fica pública	OK
US08 Público	RF08, RNF04	—	—	sem token 200 · não expõe senha/e-mail	OK
US09 Histórico	RF10, RNF06	RN07	montarPeriodo (4) · gerarCsv/escapar (2)	filtra por ponto e tipo · data inválida 400 · estatísticas · CSV text/csv	OK
US10 Relatórios	RF11	RN07	—	engenheiro 403 · totais e % de resolução	OK
US11 Usuários	RF12, RNF01	RN06	validarUsuario (2)	registro público bloqueado 403 · admin cria engenheiro · engenheiro não lista usuários	OK
Infra / API	RNF07, RNF08	—	—	rota /api inexistente responde JSON 404 · /api/saude	OK



Obrigado!

SIMSEP — Sistema de Monitoramento e Controle de Segurança em Pedreiras

Gabriel Henrique Damaceno Alves

Universidade Estadual de Goiás — UEG Trindade

github.com/7121895128/simsep